ATOMQUEST HACKATHON 1.0
GRAND FINALE

Problem Statement — Real-Time Video Support Platform

1. Background & Problem Context

Customer support teams today rely heavily on voice calls to resolve product issues. This works for simple queries, but falls apart the moment the issue requires visual context — a field engineer troubleshooting a device, an agent walking a customer through a UI, or a technician verifying a physical installation. The agent is flying blind, the customer is frustrated, and the call ends without resolution.

A video call layer solves this. But bolting on a third-party video SDK creates dependency, limits control, and keeps sensitive support interactions on someone else's infrastructure.

The challenge is to build a real-time video calling platform — owned and operated entirely by you — that a customer support team can use to conduct, record, and review video-assisted support sessions.

2. What You Need to Build

Participants must design and develop a functional video calling system with the following core capabilities.

2.1 Session Management (Must-Have)

- A support agent can create a call session and invite a customer via a shareable link or token
- Both parties can join the session from a web browser — no app installation required
- The system must track who is in a session at any point in time
- A session can be ended by either participant; all active connections are closed cleanly

- Session history (who joined, when, how long) must be persisted and queryable

2.2 Audio & Video Calling (Must-Have)

- Both participants can see and hear each other in real time
- **Media must route through a server — direct peer-to-peer connections are not acceptable**
- The call must remain stable under normal network conditions
- Either participant can mute their audio or turn off their video at any time

2.3 In-Call Chat (Must-Have)

- Participants can exchange text messages during an active call
- Messages are delivered in real time and persisted for the session record
- The chat history for a session must be retrievable after the call ends

2.4 User Roles & Access

The system must support two distinct roles:

Role	Responsibilities
Call Agent	Creates sessions; initiates and ends calls; start/stop recording
Customer	Joins via an invite link or token; cannot create or end sessions

Access must be enforced — a customer must not be able to perform agent actions, and joining a session must require a valid invite.

3. Good-to-Have Features (Bonus Points)

The following enhancements are not mandatory but will be positively evaluated. Participants that implement any of these will earn additional credit.

3.1 Call Recording

The agent can start and stop a recording during a session. Once the call ends, the recording is processed and made available for download. The system must indicate recording status (in progress / processing / ready) and provide a way to retrieve the final file.

3.2 File Sharing in Chat

Participants can upload and share files (images, PDFs, documents) within the chat during a call. Files are stored securely and accessible via the session record after the call.

3.3 Reconnect Handling

If a participant's connection drops unexpectedly, the system holds their session state for a short grace window. If they reconnect within that window, they re-enter the call seamlessly without other participants being notified of the drop. After the window expires, they are treated as having left.

3.4 Admin Dashboard

A web-based interface for operations teams showing: live sessions with participant details and duration, session history with event logs, and the ability to end any active session.

3.5 Observability

The system exposes operational metrics (active sessions, connected participants, error rates) in a format compatible with standard monitoring tools.

4. Evaluation Parameters & Scoring

Submissions will be judged by a panel against the following criteria. Each parameter carries equal weight unless otherwise announced on the day.

#	Parameter	What Evaluators Will Look For
1	**Functionality**	Does the video call work end-to-end? Can an agent create a session, a customer join, and both see and hear each other without errors?
2	**Reliability**	Does the system handle edge cases gracefully — unexpected disconnects, duplicate joins, invalid invites — without crashing or leaving inconsistent state?
3	**Architecture**	Is the design sensible and scalable? Are concerns separated cleanly? Would the system hold up under multiple concurrent sessions?

#	Parameter	What Evaluators Will Look For
4	**User Experience**	Is the interface usable by a non-technical customer without guidance? Are error states communicated clearly?
5	**Good-to-Have Features**	Has the participants implemented any features from Section 3? Depth and quality of implementation will be assessed.
6	**Code Quality**	Is the code readable and structured? Are there obvious security issues — open endpoints, insecure file handling, or broken access control?

5. Constraints & Ground Rules

- Participants may use any technology stack (language, framework, database, cloud provider)
- The solution must be accessible via a web browser — no desktop-only applications
- Third-party hosted video APIs (Twilio Video, Agora, Daily, Vonage, etc.) are not permitted — media must route through your own server
- A working demo with a complete end-to-end call (agent creates session → customer joins → both on video → call ends) must be presented, share a screen-recorded video of this demo if you are not hosting this online.
- Code must be version-controlled and the repository link submitted before the deadline
- Participants must provide a brief architecture diagram/write-up explaining their technology and system design choices

6. Submission Deliverables

- Live demo URL or locally runnable build
- Source code repository (GitHub / GitLab / Bitbucket)

- Architecture diagram (PDF or image)
- Login credentials or a way to switch between agent and customer roles during judging
- A short README covering setup steps and any known limitations